How Configuration and Deployment Changes Trigger Large Cloud and SaaS Outages: An Empirical Study of 70 Public Postmortems

Pujitha Sri Lakshmi Paladugu
Independent Researcher, Lynnwood, Washington, USA
pujitha.rathna@gmail.com · ORCID 0009-0001-5678-4022

Abstract

Context. Cloud and software-as-a-service providers publish postmortems after large outages, and many name a change as the trigger, but this evidence has rarely been coded systematically. **Objective.** We characterise how changes trigger large outages: which change artifacts dominate, how rollout scope and missing safeguards relate to duration and blast radius, how such outages are detected and mitigated, and which validation gaps recur. **Method.** We coded 70 incidents from 22 providers (2019–2026) using only official vendor postmortems, analysed them with non-parametric tests and effect sizes, and synthesised the stated validation gaps into a ten-theme taxonomy using a reproducible keyword mapping with recorded overrides. A seeded rule-based re-coding of 20 incidents checked the consistency of the trigger codes. **Results.** Changes started 52 of 70 incidents (74.3%; 95% confidence interval 63.0–83.1%). Configuration changes alone accounted for 28 (53.8% of change-triggered incidents), ahead of infrastructure changes (16) and code deployments (8). Changes pushed globally at once produced wide impact in 10 of 11 cases. Changes applied to a single cluster or site still caused impact beyond that site in 10 of 11 cases. Only 19 of 52 change-triggered incidents were resolved by rollback. The most frequent gaps were testing that did not represent production conditions, missing or bypassed staged rollout, and missing semantic validation of configuration and data. Rule-based re-coding agreed with the original codes at $\kappa = 0.87$. **Conclusions.** Configuration and data need the same progressive, validated delivery as code. We derive an eleven-control Safe Change Checklist traced to the motivating incidents.

Keywords: configuration changes; incident postmortems; cloud outages; safe deployment; continuous deployment; site reliability engineering

1 Introduction

Large cloud and software-as-a-service (SaaS) outages are among the most visible failures of modern software engineering. When a content delivery network, a cloud region or a widely used

developer platform fails, the impact reaches far beyond the provider's own customers. In recent years, several of the most consequential outages were not caused by hardware failure or attack. They were caused by the provider's own changes: a firewall rule pushed to every edge server, a content update shipped to every endpoint agent, a database permission change that silently doubled the size of a generated file, or a maintenance command that withdrew a backbone from service. Practitioners have long suspected that change is the main source of production risk, and the site reliability engineering (SRE) literature recommends progressive rollout and fast rollback for exactly this reason (Beyer et al. 2016; Warner et al. 2018).

Software engineering research has studied configuration errors in depth, mostly at the level of individual systems (Yin et al. 2011; Xu and Zhou 2015; Sayagh et al. 2020). It has also studied production incidents inside individual organisations (Liu et al. 2019; Ghosh et al. 2022; Ganatra et al. 2023) and outages across services as reported in news coverage and public postmortems (Gunawi et al. 2016; Li et al. 2022). Less is known, quantitatively and across providers, about the *mechanics of change-triggered outages in the current era of continuous deployment*. We do not know which kinds of artifact are changed, how far the change had spread when it failed, whether the impact stayed within the scope of the change, how the outage was ended, and which safeguards the providers themselves say were missing. Public postmortems are an imperfect but valuable source for these questions. They are written by the people who ran the response, they often state the missing safeguard explicitly, and anyone can read and re-code them.

This paper reports an empirical study of 70 large outages at 22 cloud and SaaS providers between 2019 and 2026, each documented by an official vendor postmortem. We coded each incident for its initiating event, the changed artifact, rollout scope at failure, detection, mitigation, duration, blast radius and the validation gap stated by the source. We then analysed these codes quantitatively and thematically. We make four contributions:

1. **A coded, public dataset and replication package.** The dataset contains 70 incidents with source URLs, a documented codebook, and recorded ambiguities and alternative codings. Derived codings, the analysis script and every reported statistic are released with the paper (Section 4).
2. **Quantitative characterisation of change-triggered outages** (RQ1–RQ3). Changes start most of these outages, and configuration is the dominant changed artifact. Global,

simultaneous pushes are associated with wide impact. Locally scoped changes frequently cause impact far beyond their scope. Rollback ends only about a third of change-triggered outages.

3. **A taxonomy of ten validation-gap themes** (RQ4). The themes are derived from the safeguards the providers themselves say were missing, using a reproducible keyword mapping with recorded manual overrides.
4. **A Safe Change Checklist** of eleven controls. Each control is traced to the themes and incidents that motivate it, with counts, so that practitioners can weigh each control by evidence rather than by convention (Section 7).

The rest of the paper is organised as follows. Section 2 reviews related work. Section 3 states the research questions, and Section 4 describes the data, coding and analysis. Section 5 reports the results for each research question. Section 6 discusses the findings, Section 7 presents the checklist, Section 8 discusses threats to validity, and Section 9 concludes.

2 Background and Related Work

2.1 Configuration errors and change-induced failures

Configuration has been recognised as a major source of failure for more than a decade. Yin et al. (2011) characterised several hundred real configuration errors from a commercial storage system and four open-source systems. They found that most were mistakes in parameter values, and that many led to severe impact while giving the user little diagnostic help. Xu and Zhou (2015) surveyed systems approaches to misconfiguration across prevention, detection, diagnosis and repair. Later work moved the defence earlier in the lifecycle. Xu et al. (2016) generated checkers that exercise configuration values at initialisation, so that latent errors surface before they can cause damage. Sun et al. (2020) proposed testing configuration changes *in context* by running existing code tests with the values about to be deployed. Mehta et al. (2020) mined correlated changes across code and configuration to flag changes that are likely to be incomplete. Sayagh et al. (2020) combined interviews, a survey and a literature review to describe configuration engineering as practised. They documented the weak tooling and process support that practitioners experience. On the industrial side, Tang et al. (2015) described holistic configuration management at a large social network, with configuration treated as code, reviewed, canaried and automatically

validated. Maurer (2015) listed rapidly deployed configuration changes among the most common causes of failure at that company.

Our study complements this line of work with a different unit of analysis. Rather than individual configuration errors in individual systems, we study *outages at provider scale* triggered by changes of any kind. We ask how configuration compares with code and infrastructure changes as a trigger, and which safeguards were missing when those changes escaped.

2.2 Studies of cloud incidents and outages

Gunawi et al. (2014) studied about three thousand issues from the bug trackers of open-source cloud systems. Gunawi et al. (2016) then analysed hundreds of service outages from news reports and public postmortems over several years, reporting root causes, impact and recovery. They identified software upgrades and configuration among the leading root-cause categories. Yuan et al. (2014) analysed user-reported failures in distributed data-intensive systems. They showed that most catastrophic failures stemmed from incorrect handling of non-fatal errors, and that simple testing of error-handling code would have prevented many of them. Zhang et al. (2021) studied upgrade failures in distributed systems, many of which arose from incompatible data or state across versions. Bronson et al. (2021) introduced metastable failures, which persist after their trigger has been removed. Huang et al. (2022) studied such failures in publicly reported incidents. Huang et al. (2017) described gray failure, in which the system and its observers disagree about health. This is a recurring theme in large outages and is relevant to how outages are detected.

Studies inside providers offer depth that public data cannot. Liu et al. (2019) analysed production incidents at a large cloud provider and characterised the software bugs behind them, including bugs in how components handle data produced by other components and in fault handling. Ghosh et al. (2022) published an empirical study of a year of production incidents in a large-scale cloud service. It examined their root causes and how they were detected, triaged and mitigated. Ganatra et al. (2023) examined incidents from the perspective of detection, asking why monitors failed to catch problems earlier. Li et al. (2020) described an analytics service that correlates deployment events with failure signals to stop bad rollouts in a large cloud infrastructure. These studies are rich but confined to one organisation, and their data cannot be shared.

2.3 Empirical studies of public postmortems

A smaller body of work studies public incident reports directly. Li et al. (2022) analysed 354 public postmortems from three major cloud providers and derived guidelines for each stage of the fault life cycle. Sillito and Kutomi (2020) studied publicly posted incident reports to understand how software teams detect, diagnose and fix failures. Chu et al. (2025) characterised outages and incidents of public large-language-model services from their status pages. Community collections, such as the list of postmortems curated by Luu (n.d.), make these reports easier to find but do not code them. Our study differs in its focus and coding. We restrict the corpus to official vendor sources and to incidents with material, multi-service or provider-wide impact. We code change-specific attributes (artifact, rollout scope at failure, mitigation by rollback) that prior studies do not code. We also synthesise the *stated* validation gaps into a taxonomy that is linked to concrete controls.

2.4 Safe deployment practice

Practitioner literature describes the safeguards whose absence we measure. The SRE book treats release engineering as a discipline of reproducible, reviewed and staged releases (Beyer et al. 2016, ch. 8). The SRE workbook treats configuration design as a source of reliability risk (Davidovič et al. 2018) and describes canarying as a statistical comparison between a small exposed population and a control (Warner et al. 2018). Liguori (2020) describes a deployment pipeline organised in waves of increasing size, with bake times, automated monitors and automatic rollback. These sources describe what good practice looks like. Our study asks how often the providers' own postmortems say that such practice was missing, bypassed or ineffective when a large outage occurred.

3 Research Questions

We pose four research questions.

RQ1. How often are large outages triggered by changes, and which change artifacts dominate? We estimate the share of incidents started by a change and compare configuration, code and infrastructure changes. Within configuration, we identify which kinds of artifact were changed. This question establishes whether change, and configuration in particular, deserves the attention that practitioners give it.

RQ2. How do rollout scope at failure and validation gaps relate to outage duration and blast radius? Progressive rollout is meant to limit the blast radius of a bad change. We ask whether outages from globally pushed changes differ from outages whose change was staged, regional or local, both in how long they lasted and how far their impact spread. We also ask whether the number of validation gaps relates to duration.

RQ3. How are change-triggered outages detected and mitigated, and is there a relationship with duration? Rollback is widely treated as the default remedy for a bad change. We ask how often it actually ends change-triggered outages, what is done instead, and whether the mitigation path or the detection source is associated with shorter outages.

RQ4. Which validation gaps recur in change-triggered outages, and how can they be organised? We synthesise the safeguards that sources say were missing into a taxonomy and count incidents per theme. The taxonomy is the basis for the practitioner checklist in Section 7.

4 Methodology

The study is an exploratory, observational analysis of archival documents. Following the guidance for case-based and archival research in software engineering (Runeson and Höst 2009; Ralph et al. 2020), we describe the data source, the selection, the coding, the derived codes, the reliability check and the statistical procedures in enough detail to be repeated.

4.1 Data source and selection

The unit of analysis is an incident: a period of material customer impact with a single initiating event. Incidents were collected from official vendor postmortems, namely engineering blogs, status-page incident reports, provider post-event summaries, post-incident reviews and availability reports. News coverage and third-party status aggregators were not used. Inclusion required that (i) the postmortem be published by the affected vendor; (ii) customer impact started between 1 January 2019 and the collection date, 5 October 2026; (iii) the source describe the initiating event in enough detail to code it; and (iv) the impact be material, meaning multi-service, multi-region or provider-wide, or a full outage of a major product. Where one report covered several incidents, each incident received its own row. Reports that could not be retrieved, had been removed, or remained in a preliminary state without a final review were excluded, and the exclusions are listed in the dataset documentation. The final dataset has 70 incidents from 22 providers. The four most

frequent providers, Cloudflare (18), Azure (12), Google Cloud (8) and AWS (7), together contribute 45 incidents. The remaining 25 come from 18 SaaS, developer-platform, security and consumer-internet providers. Incidents per year range from 3 (2019) to 15 (2025).

4.2 Coding scheme

Each incident was coded on the fields in Table 1. The codebook follows a conservatism rule: when a source is ambiguous, the most literal reading wins and the alternative is recorded in a free-text notes field. Two consequences of this rule matter for interpretation. First, the *initiating* event is coded, so a configuration change that exposed a latent code defect is coded as a configuration change. Second, a kill switch, a restart or a restore from backup is coded as manual reconfiguration rather than rollback. Rollback is reserved for reverting the triggering change or restoring the state that existed before it. Durations were taken from the source or computed from its stated start and resolution timestamps. Where a stated figure conflicted with the timeline, the stated figure was used and the conflict noted.

Table 1. Coding scheme (abridged; the full codebook is in the replication package).

Field	Values and rule
trigger_category	Initiating event: config_change (configuration, rules, policy, feature flags, routing, quota, DNS records, or data/content/metadata files); code_deploy (software release or schema migration); infrastructure_change (maintenance, capacity addition, automated scaling, OS/platform upgrade); capacity_exhaustion; dependency_failure; hardware_or_network_fault; security_or_cert_expiry; other
change_related	yes if a human or automated change started the incident (including automated scaling and credential rotation); otherwise no
change_artifact	Free text naming what was changed (e.g. feature flag, BGP policy, data file, binary, schema, maintenance command)
rollout_scope_at_failure	global_simultaneous; staged_but_escaped (a staged rollout existed but the defect passed or bypassed it); regional; single_cluster (one cluster, data centre, zone, cell or device); not stated
detection	automated_alert; customer_report; engineer_observation; not stated
mitigation	rollback; forward_fix; failover; manual_reconfig (kill switches, restarts, throttling, traffic engineering, rebuilds, restores); not stated
validation_gap	Missing safeguard as stated by the source, otherwise not stated
duration_minutes, blast_radius, notes	Stated or derived duration; scope of impact; ambiguities and alternative codings

4.3 Derived codings

Three derived codings support the analysis. They are released as separate files so that they can be inspected or replaced. **Configuration sub-type.** For the 28 configuration changes, the free-text change artifact was mapped by an ordered regular-expression list to six sub-types: data, content or

metadata files; routing, BGP or DNS; firewall, WAF or allow-list rules; feature flags or feature enablement; quota, permission, credential or administrative actions; and service, tenant or customer configuration. **Blast-radius level.** The free-text blast radius was coded by the author into an ordinal level. Level 3 means global, multi-region, or most or all users of the affected product. Level 2 means one region, with any number of services. Level 1 means one zone, cell, cluster or device, or a stated minority subset of users, requests or customers. Primary scope was coded, so limited spillover into a second region did not raise the level. **Alternative codings.** The notes field records 13 incidents for which the source supports a different trigger category, for example a configuration change that exposed a latent code defect. These are used in a sensitivity analysis.

4.4 Thematic synthesis of validation gaps

For RQ4 we followed the steps of thematic synthesis in software engineering (Cruzes and Dybå 2011), which adapt thematic analysis (Braun and Clarke 2006) to evidence from documents. We first read all stated validation gaps and notes and grouped the safeguards they name into candidate themes. We then refined the themes until each was distinct and could be described in one sentence. This produced the ten themes in Table 4. To make the coding reproducible, each theme was operationalised as a regular expression applied to the validation_gap field (released as theme_keyword_map.csv). Keyword matching alone is imperfect, so we reviewed every incident and recorded 19 manual overrides, each with its justification, in the coding file (validation_gap_themes.csv). Overrides either remove a keyword match that refers to something else (for example, "cell" naming the system under test rather than a missing cell boundary) or add a theme supported by the notes field (for example, a note that the automated rollback failed because it depended on the connectivity that had been lost). An incident can belong to several themes. Themes were counted for all incidents and separately for the 52 change-triggered incidents.

4.5 Reliability check of the trigger codes

The original coding was done by a single coder, which is a known limitation of the dataset. A second human coder was not available for this independent study. As a partial substitute, we re-derived the trigger category for a random sample of 20 incidents (NumPy generator, seed 20261005) with an independent rule-based coder. The coder scores each category by counting category-specific cue words in the title (weight 2) and in the notes and validation-gap text (weight 1). It picks the highest-scoring category and breaks ties in favour of change categories, as the

codebook requires. The rules were written once from the codebook definitions and were not tuned on the sample. We report Cohen's κ (Cohen 1960) between the original and re-derived codes, with a bootstrap confidence interval (2,000 resamples), and also κ over all 70 incidents. We are explicit about what this check is not. The rule-based coder was written by the same author who designed the codebook. It reads a notes field written by the original coder, which sometimes names alternative categories. So it measures how consistently the codebook can be applied to the recorded text. It does not measure agreement between independent human judges, and it probably overstates agreement relative to a true second coder.

4.6 Statistical analysis

Durations are heavily right-skewed: the longest outage lasted 14 days and the shortest 19 minutes. We therefore report medians with interquartile ranges (IQR) and use rank-based tests. The Mann–Whitney U test (Mann and Whitney 1947) compares two groups and is reported with the rank-biserial correlation r (Kerby 2014). A negative r means the first-named group tends to have shorter durations. The Kruskal–Wallis test (Kruskal and Wallis 1952) compares more than two groups and is reported with ε^2 (Tomczak and Tomczak 2014). Associations between categorical variables use Fisher's exact test with odds ratios. Proportions are reported with Wilson score intervals (Wilson 1927). Incidents whose duration or category is not stated are excluded from the corresponding test, and the number of incidents in each group is reported throughout. Because the groups are small and several tests are run, we treat all tests as exploratory. We apply the Holm correction (Holm 1979) across the family of eleven hypothesis tests reported in Sections 5.2–5.5 and give both raw and adjusted p-values. We follow the recommendation to prefer robust, effect-size-centred reporting in empirical software engineering (Kitchenham et al. 2017). Analyses use Python with pandas, NumPy, SciPy (Virtanen et al. 2020) and Matplotlib.

4.7 Use of AI-assisted tools

In line with Springer Nature's policy, the author discloses that an AI assistant (Claude, Anthropic) was used to help draft and edit the text and to write and debug the analysis code. The dataset was coded from public sources as documented in the replication package, and every reported number is produced by the released analysis script. The author designed the study, reviewed and verified all content, codes, code and results, and takes full responsibility for the article.

4.8 Replication package

The replication package contains the dataset and its documentation, the derived coding files (thematic coding with keyword matches and overrides, configuration sub-types, blast-radius levels, alternative codings and the reliability sample), the analysis script, all figures, and a results file with every statistic reported here. Running the script regenerates all derived files, figures and results.

5 Results

5.1 RQ1: Changes as triggers

Figure 1(a) shows the distribution of initiating events. A change started 52 of the 70 incidents (74.3%; 95% CI 63.0–83.1%). Configuration changes were the largest category with 28 incidents (40.0% of all incidents; 53.8% of change-triggered incidents, 95% CI 40.5–66.7%). Infrastructure changes followed with 16 (30.8% of change-triggered incidents), then code deployments with 8 (15.4%). Among the 18 incidents not started by a change, hardware or network faults were the most common (8). Dependency failures and security or certificate events contributed 4 each, and capacity exhaustion without a precipitating change contributed 2.

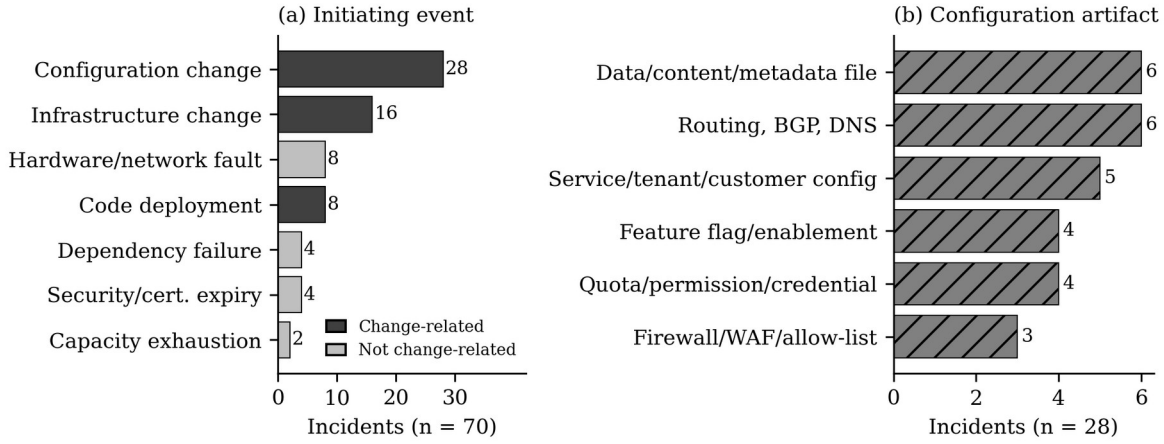


Fig. 1 (a) Initiating events of the 70 incidents; dark bars are change-related. (b) Changed artifact within the 28 configuration-change incidents.

Configuration changes covered a wide range of artifacts (Fig. 1b). There were data, content or metadata files (6), routing, BGP or DNS changes (6), service, tenant or customer configuration (5), feature flags or feature enablement (4), quota, permission, credential or administrative actions (4), and firewall, WAF or allow-list rules (3). Data and metadata files are notable. In the endpoint-agent content update of July 2024 (CrowdStrike 2024-07-19), the globally replicated policy data

of June 2025 (Google Cloud 2025-06-12), the edge metadata incidents of October 2025 (Azure 2025-10-09, Azure 2025-10-29) and the generated feature file of November 2025 (Cloudflare 2025-11-18), the changed artifact was not hand-written configuration. It was data produced by another system and consumed by the data plane. In several of these incidents, the configuration change exposed a defect that had shipped earlier in code. This is why the codebook's choice to code the initiating event matters.

Robustness. Three checks suggest that the headline proportions do not depend on any single coding decision. First, applying the 13 alternative codings recorded in the notes moves the number of configuration-triggered incidents within a range of 22 to 30 (31.4%–42.9% of all incidents). With every alternative applied at once, configuration remains the largest category (24 incidents) and 52 incidents remain change-related. Second, excluding the two most frequent providers leaves 31 of 40 incidents change-related (77.5%; 95% CI 62.5–87.7%). The configuration share falls to 30.0%, so the prominence of configuration is partly, but not wholly, a property of the providers that publish most. Third, the change-related share did not differ between 2019–2022 (19 of 25) and 2023–2026 (33 of 45; Fisher $p = 1.000$), and neither did the configuration share (10 of 25 versus 18 of 45; $p = 1.000$).

Duration by trigger. Change-triggered outages lasted a median of 281 min (IQR 68–681; $n = 48$). Other outages lasted a median of 414 min (IQR 201–1,708; $n = 16$). The difference was not statistically significant (Mann–Whitney $p = 0.156$, Holm-adjusted 0.754; $r = -0.24$). Among change types, configuration-triggered outages had the shortest median, 180 min (IQR 60–497; $n = 27$). Infrastructure changes had 297 min (IQR 142–1,028; $n = 13$) and code deployments 390 min (IQR 135–660; $n = 8$), but the differences were small and not significant (Kruskal–Wallis $H = 1.49$, $p = 0.476$, $\varepsilon^2 = 0.03$). Hardware and facility faults, typically cooling or power failures, had the longest median, 1,931 min (IQR 1,003–2,384; $n = 7$).

Answer to RQ1. About three in four large outages in the corpus were started by a change (52/70). Configuration was the most frequent changed artifact (28/52), ahead of infrastructure (16) and code (8). Data and metadata files produced by other systems form a distinct and recent sub-group.

5.2 RQ2: Rollout scope, blast radius and duration

Rollout scope at failure was stated for 37 of the 52 change-triggered incidents (Table 2). The change had been pushed globally at once in 11. In 10, a staged rollout existed but the defect passed

or bypassed it. In 5, the change was applied to a single region, and in 11 to a single cluster, site, zone, cell or device. Scope differed by change type. Configuration changes were mostly global (7) or staged but escaped (8). Infrastructure changes were mostly single-cluster (6) or regional (3).

Table 2. Change-triggered incidents by rollout scope at failure: duration, blast-radius level and rollback.

Rollout scope at failure	n	Duration, min: median (IQR)	Blast level L1 / L2 / L3	Ended by rollback
Global, simultaneous	11	59 (27–180); n = 9	1 / 0 / 10	6 of 11
Staged but escaped	10	490 (346–712); n = 9	2 / 1 / 7	4 of 10
Regional	5	872 (670–1,028); n = 5	0 / 5 / 0	2 of 5
Single cluster or site	11	265 (62–374); n = 11	1 / 4 / 6	3 of 11
Not stated	15	189 (85–428); n = 14	5 / 0 / 10	4 of 15

Duration is computed over incidents with a stated duration (n shown). Blast level: L1 = zone, cell, cluster, device or minority subset; L2 = one region; L3 = global, multi-region, or most or all users.

Duration. Figure 2 shows the durations. Outages from globally pushed changes had the shortest median, 59 min (IQR 27–180; n = 9). Outages from changes that had passed a staged rollout had a median of 490 min (IQR 346–712; n = 9). Regional changes had the longest median, 872 min (IQR 670–1,028; n = 5). Across the four stated scopes the differences did not reach significance (Kruskal–Wallis H = 6.93, p = 0.074, Holm-adjusted 0.519), although the effect size was moderate ($\varepsilon^2 = 0.21$). The direct comparison between global-simultaneous and staged-but-escaped changes gave a large effect in favour of shorter global outages (r = -0.51; U = 20, p = 0.077, Holm-adjusted 0.519; n = 9 and 9). This effect is not robust to provider composition. With Cloudflare's incidents excluded it shrank to r = -0.26 (p = 0.530; n = 5 and 7), because short, globally pushed edge-configuration incidents come mostly from one provider. The data do not support the reading that staged rollout makes outages longer. They are consistent with a more modest observation: when a defect escapes a staged rollout, it tends to be one that is slow to manifest or hard to attribute, and the resulting outage is not short.

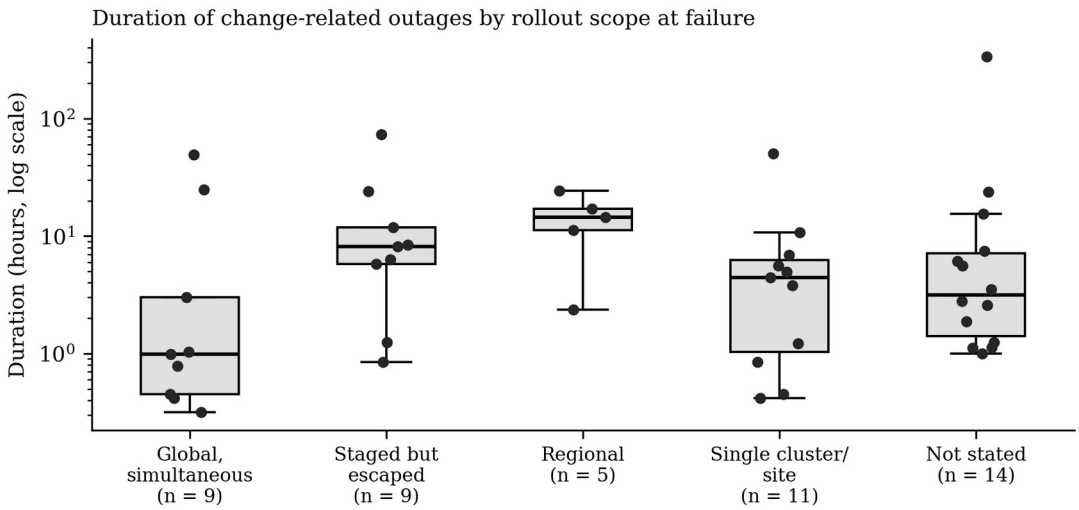


Fig. 2 Duration of change-triggered outages by rollout scope at failure (box: IQR; line: median; points: incidents; log scale). Incidents without a stated duration are omitted.

Blast radius. Globally pushed changes produced level-3 (global, multi-region or most-users) impact in 10 of 11 cases. Changes with any other stated scope did so in 13 of 26 (odds ratio 10.0, Fisher p = 0.027, Holm-adjusted 0.244). The more instructive result concerns local changes. Of the 11 changes applied to a single cluster, site or device, 10 caused impact beyond that unit (level 2 or 3), and 6 caused global or most-users impact. In these incidents, the change touched one location, but a protocol, control plane or shared dependency carried its effect much further. Examples are a maintenance job intended for one location that descheduled network control-plane jobs in several (Google Cloud 2019-06-02), a routing change at one site that attracted backbone traffic from many (Cloudflare 2020-07-17), and a command on one router whose routing-protocol effect propagated across a global wide-area network (Azure 2023-01-25). The scope of a change is therefore a poor proxy for the scope of its possible impact.

Duration and blast radius. Across all incidents with a stated duration, duration differed by blast level (Kruskal–Wallis H = 10.35, p = 0.006, $\varepsilon^2 = 0.16$; Holm-adjusted 0.062). The pattern is not monotonic. Regional (level-2) outages had a median of 792 min (IQR 353–1,708; n = 16), against 195 min (IQR 68–481; n = 36) for level 3 and 190 min (IQR 58–390; n = 12) for level 1. Level-2 incidents are almost all regional events at the large infrastructure providers (15 of 16). In these events, recovery involved many dependent services, queued work and stateful systems, and the postmortems often measure duration to the last recovered service. Wide outages, by contrast, are often both obvious and simple to reverse.

Validation gaps and duration. Among change-triggered incidents with at least one coded theme and a stated duration, the number of themes was not associated with duration (Spearman $\rho = 0.15$, $p = 0.322$, $n = 46$).

Answer to RQ2. Global, simultaneous pushes were strongly associated with wide impact. Staged rollouts that the defect escaped were followed by longer outages in this sample, but the difference was not significant and depends on provider mix. Most strikingly, 10 of 11 single-site changes caused impact beyond the site. Neither rollout scope nor the number of validation gaps reliably predicted duration once multiple testing was taken into account.

5.3 RQ3: Detection and mitigation

Detection. The detection source was stated for 49 of the 70 incidents. Of these, 38 were first detected by automated alerting (77.5%; 95% CI 64.1–87.0%), 6 by engineers' observation and 5 by customer reports. Among change-triggered incidents with a stated source, 29 of 36 were detected by alerts. Large outages are mostly detected automatically. The postmortems indicate that the difficulty usually lay in attributing the symptoms to the change and reversing it, not in noticing them. Durations did not differ between automatically and humanly detected incidents (median 335 min for alerts, $n = 35$, versus 11 human-detected incidents; $p = 0.487$, $r = -0.14$; Fig. 3b).

Mitigation. Only 19 of the 52 change-triggered incidents (36.5%; 95% CI 24.8–50.1%) were ended by rolling back the change. Manual reconfiguration ended 30, failover 2 and a forward fix 1. Rollback was most common for configuration changes (12 of 28), less common for code deployments (3 of 8), and least common for infrastructure changes (4 of 16). Maintenance, scaling and upgrades often cannot be undone simply, or reversing them does not restore lost state. The postmortems record several reasons why rollback was not the remedy. In 5 change-triggered incidents, containment came from a kill switch or from disabling the offending component rather than reverting the change (Cloudflare 2019-07-02; Cloudflare 2020-07-17; Google Cloud 2020-12-14; Azure 2024-01-21; Google Cloud 2025-06-12). In others, the change had already destroyed or corrupted state, so service returned only after rebuilds or restores (Atlassian 2022-04-05; AWS 2025-10-19). In others again, reverting made matters worse or could not reach the affected systems. Examples include a DNS signing change whose reversal deepened the failure (Slack 2021-09-30), endpoints that had crashed before the revert and needed individual remediation

(CrowdStrike 2024-07-19), and an automated rollback that depended on the network connectivity the change had removed (Azure 2026-07-23).

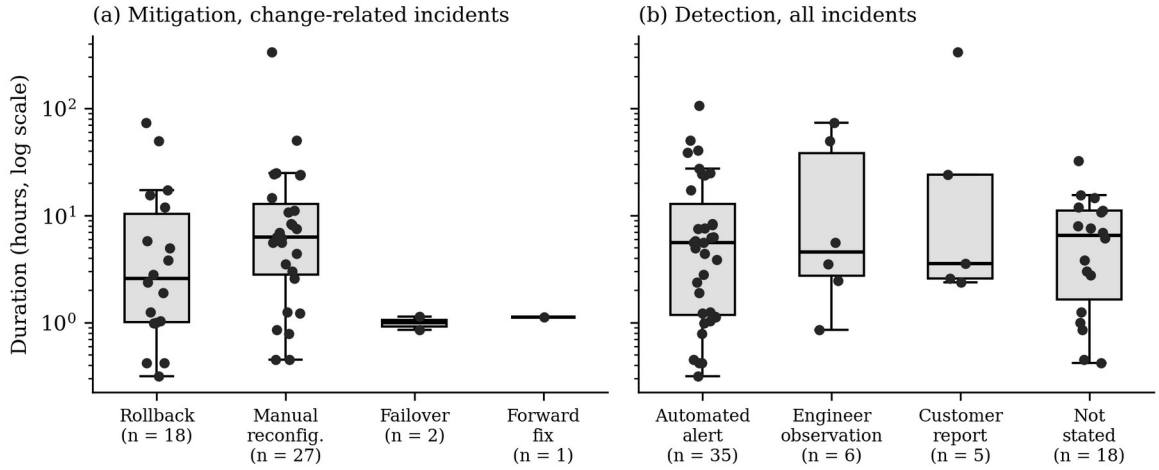


Fig. 3 Duration by (a) mitigation, for change-triggered incidents, and (b) detection source, for all incidents (log scale).

Mitigation and duration. Rollback-mitigated change-triggered outages had a median duration of 155 min (IQR 61–621; n = 18). Those ended by manual reconfiguration had a median of 377 min (IQR 168–771; n = 27) (Fig. 3a). The difference is in the expected direction with a small-to-medium effect (r = -0.26), but it was not significant (U = 180.5, p = 0.151, Holm-adjusted 0.754). Rollback-mitigated outages include some of the longest in the corpus, for example a 73-hour platform outage in which a second cause also had to be resolved (Roblox 2021-10-28). Rollback is therefore not by itself a guarantee of short outages.

Answer to RQ3. Change-triggered outages were usually detected by automated alerts, but only about a third were ended by rollback. The remainder required kill switches, manual reconfiguration, failover or state restoration. Rollback-mitigated outages tended to be shorter, but not significantly so in this sample.

5.4 RQ4: A taxonomy of validation gaps

A validation gap was stated for 56 of the 70 incidents. The rate was much higher for change-triggered incidents (50 of 52) than for others (6 of 18). This is consistent with postmortems of change-triggered incidents focusing on the safeguard that should have stopped the change. After keyword mapping and overrides, 50 of the 52 change-triggered incidents were assigned at least

one theme. The median was 2 themes per incident (IQR 1–3; maximum 4), and 27 incidents had two or more. Table 3 defines the ten themes and Fig. 4 shows their frequency.

Table 3. Taxonomy of validation gaps. Counts are incidents with the theme among change-triggered incidents (n = 52) and among all incidents (n = 70); an incident can have several themes.

ID	Theme and definition	Change / all	Example incidents
T1	**Staged rollout missing, bypassed or not health-gated.** The change reached the fleet without stages, bypassed them, or advanced without health signals	16 / 16	Cloudflare 2019-07-02; CrowdStrike 2024-07-19; Cloudflare 2025-12-05
T2	**No semantic validation before propagation.** Configuration, data or operation inputs were not checked for meaning, size, staleness or compatibility	13 / 14	Google Cloud 2021-11-16; AWS 2025-10-19; Cloudflare 2025-11-18
T3	**Missing blast-radius limits.** No bound on how many locations, cells, prefixes or hosts one change could affect	7 / 8	Google Cloud 2019-06-02; AWS 2023-06-13; Azure 2026-07-23
T4	**Rollback or fallback missing, slow or ineffective.** No automated revert, or the revert or last-known-good path failed or made things worse	9 / 9	Cloudflare 2022-06-21; Slack 2021-09-30; Azure 2025-10-29
T5	**Testing not production-representative.** Tests, benchmarks or staging did not exercise the data, scale, version mix or code path that failed	19 / 22	Roblox 2021-10-28; Linear 2024-01-24; Google Cloud 2025-06-12
T6	**Ungoverned change or weak guardrails.** The change came through a manual or automated channel outside the governed pipeline, or tooling did not block a dangerous action	11 / 11	Meta 2021-10-04; Datadog 2023-03-08; Cloudflare 2025-02-06
T7	**Monitoring gap.** No signal existed, or it was broken, for the failure mode the change triggered	5 / 7	AWS 2020-11-25; PagerDuty 2025-08-28
T8	**Fragile consumer.** The receiving component crashed or degraded on malformed or oversized input instead of rejecting it	4 / 5	Discord 2020-03-16; Google Cloud 2025-06-12
T9	**Capacity or scaling assumptions.** The change crossed an untested scale threshold or removed headroom	7 / 9	AWS 2023-06-13; GitHub 2026-08-06
T10	**Hidden coupling or dependency.** Recovery, monitoring or the data plane depended on the system being changed or on a hidden shared component	4 / 7	Roblox 2021-10-28; Azure 2026-07-23

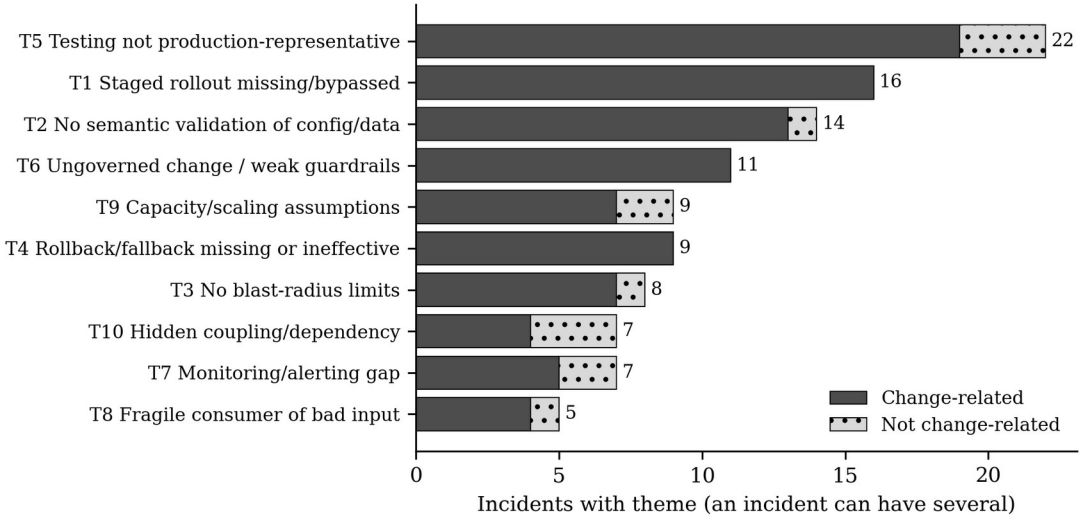


Fig. 4 Frequency of validation-gap themes, split by whether the incident was change-triggered.

The most frequent theme among change-triggered incidents was **T5, testing that did not represent production conditions** (19 of 52). This theme captures defects that existed before the change and were only exercised by it. Examples are a code path never reached during a staged binary rollout and later activated by globally replicated policy data, a scale threshold never reached in testing, and a schema migration never tested against the code that was actually deployed. **T1, missing, bypassed or non-health-gated staged rollout** (16), and **T2, missing semantic validation of configuration or data** (13), followed. T1 and T2 were also the most frequent pair (5 incidents). This is the familiar pattern of an invalid input that was both unchecked and delivered everywhere at once. **T6, ungoverned change or weak guardrails** (11), groups manual commands that bypassed review or audit tooling with automated channels, such as operating-system update feeds, that delivered changes outside the governed pipeline. **T4, ineffective rollback** (9), records cases where the revert path was missing, slow, depended on the failed system or captured the bad state as last-known-good.

The themes are not evenly distributed across change types (Table 3 and replication package). T1 was coded for 13 of 28 configuration changes but only 3 of 24 code and infrastructure changes (Fisher $p = 0.015$, Holm-adjusted 0.147). T2 showed the same direction (10 of 28 versus 3 of 24; $p = 0.064$, Holm-adjusted 0.513). Neither survives correction, but both point the same way. When configuration triggered an outage, the postmortems disproportionately said that it had been delivered without the staging and validation that code would have received. For code deployments the dominant theme was T5 (6 of 8). For infrastructure changes, themes were spread across T5 (6),

T6 (5) and T3 (4). This reflects maintenance and scaling operations that lacked scope limits and operator guardrails.

Answer to RQ4. The stated validation gaps fall into ten themes. Three of them dominate: unrepresentative testing, missing or bypassed staging, and missing semantic validation. The last two are concentrated in configuration changes. Most change-triggered incidents had more than one gap, which suggests layered rather than single safeguards.

5.5 Reliability of the trigger codes

The seeded sample of 20 incidents contained 7 infrastructure change, 6 config change, 4 hardware or network fault, 1 dependency failure, 1 code deploy, 1 capacity exhaustion incidents by the original codes. The rule-based coder agreed with the original code for 18 of 20 (90%), giving Cohen's $\kappa = 0.87$ (bootstrap 95% CI 0.65–1.00). Over all 70 incidents, $\kappa = 0.84$ (agreement 88.6%). Both disagreements in the sample were infrastructure changes that the rule-based coder labelled as configuration changes. One was a router command issued during a capacity addition (Azure 2023-01-25). The other was a cluster switch followed by a feature-flag change (Honeycomb 2023-07-25), for which the dataset notes already record configuration as an alternative coding. On the conventional scale (Landis and Koch 1977) these values indicate high agreement. For the reasons given in Section 4.5, we read them only as evidence that the codebook can be applied consistently to the recorded text, not as inter-rater reliability. The boundary between configuration and infrastructure change is where most of the residual ambiguity lies.

6 Discussion

6.1 Configuration and data are deployed like code but validated less

The most consistent message of the corpus is that configuration, and especially machine-generated data consumed by the data plane, is a first-class deployment artifact. It is often delivered with fewer safeguards than code. Code at the providers studied is generally described as travelling through review, testing and staged rollout. Configuration and data files frequently travel through faster paths, because speed is the point. Firewall rules must respond to attacks, threat content must reach endpoints quickly, and feature files must track traffic. Several postmortems describe the same pattern: an artifact generated by one system, assumed to be well-formed, consumed without defensive checks by another, and replicated globally within minutes. This matches the industrial observation that rapidly deployed configuration is a leading cause of failure (Maurer 2015) and

the recommendation to treat configuration as code (Tang et al. 2015; Davidovič et al. 2018). The new element in the recent incidents is that the artifact is often *data* rather than human-authored configuration, and its producer is another automated system. Validation must therefore sit both at the producer, as in configuration testing (Sun et al. 2020), and at the consumer, which must reject what it cannot safely use (T8).

6.2 The scope of a change is not the scope of its impact

Progressive rollout assumes that a change applied to a small part of the system can only harm that part. The single-site incidents contradict this assumption: 10 of 11 caused impact beyond the site. Routing protocols, shared control planes, service discovery and global dependencies amplified local actions into regional or global outages. This is one reason that rollout *stages* alone are insufficient. A safe rollout plan must also bound the *reach* of each stage. It should verify, in aggregate and before execution, which paths, prefixes or dependencies the change can affect, as the over-scoped break-fix incident of 2026 illustrates (Azure 2026-07-23). Blast-radius limits (T3) and hidden coupling (T10) are the taxonomy's names for this gap.

6.3 Staging is necessary but not sufficient

Ten change-triggered incidents passed or bypassed a staged rollout, and their outages were not short. The defects that escape staging share recognisable properties. Their failure is asynchronous, slow-building or load-dependent, so health gates during the bake period see nothing. Or the defect is activated by a different artifact, such as data or a flag, from the one being staged. Or the staging policy covers some locations only at the last step. These are the conditions under which canary analysis is known to struggle (Warner et al. 2018). They argue for longer or load-aware bake periods for components with known asynchronous failure modes, for staging the *activating* artifact rather than only the code, and for making sure that the protected last-known-good state cannot be overwritten by a change that has not yet proven itself.

6.4 Rollback is not a universal remedy

The SRE literature rightly treats a fast rollback as the first response to a bad change (Beyer et al. 2016). In this corpus, however, rollback ended only 19 of 52 change-triggered outages. Infrastructure changes, which destroy or move state, were rarely reversible. Configuration changes were reversible more often, but a revert cannot repair hosts that have already crashed, data that has already been deleted, or caches and resolvers that have already absorbed bad state. In several

incidents, the rollback mechanism itself depended on the component that had failed. The practical implication is to design and rehearse non-rollback containment as deliberately as rollback: kill switches, configuration freezes, out-of-band access and state restoration (C4, C10, C11 in Section 7). Rehearsal matters because one killswitch that triggered an outage had never been exercised in that configuration (Cloudflare 2025-12-05).

6.5 Implications for research

Three research directions follow. First, tools for configuration validation have mostly targeted human-authored parameters in single systems (Xu et al. 2016; Sun et al. 2020; Mehta et al. 2020). The incidents here call for validation of machine-generated data artifacts across producer and consumer boundaries, including size, cardinality and version compatibility. Second, deployment-safety analytics (Li et al. 2020) could be extended with static reasoning about reach. Before a change runs, which paths, prefixes or dependent services could it affect? Third, public postmortems are a growing and under-used resource. A shared, multiply-coded corpus with a common codebook, building on Gunawi et al. (2016), Li et al. (2022) and this study, would allow the field to track whether these failure patterns change over time.

7 Implications for Practitioners: a Safe Change Checklist

Table 4 turns the taxonomy into a checklist of eleven controls. Each control is mapped to the theme or themes that motivate it, and its count is the number of incidents in the corpus where the source stated, or the notes recorded, that the corresponding safeguard was missing or ineffective. Control C11 additionally counts incidents that were contained by a kill switch or by disabling the offending component. The counts are not estimates of how often each control would prevent an outage. They show how often its absence appears in public postmortems of large outages, which is one input to prioritisation alongside cost and local context. Most incidents had several gaps, so the controls are layers, not alternatives.

Table 4. Safe Change Checklist. Counts are incidents motivating each control among change-triggered incidents (n = 52) and all incidents (n = 70).

ID	Control	Themes	Change / all	Example incidents
C1	Progressive, health-gated rollout for every change type, including configuration, data and metadata	T1	16 / 16	Cloudflare 2019-07-02; Slack 2022-02-22
C2	Treat generated configuration and data as untrusted input: schema, semantic and size checks before propagation	T2	13 / 14	Google Cloud 2020-12-14; Google Cloud 2021-11-16

ID	Control	Themes	Change / all	Example incidents
C3	Enforce blast-radius limits per change: cells, batch sizes, prefix caps, aggregate-scope checks	T3	7 / 8	Google Cloud 2019-06-02; Cloudflare 2020-07-17
C4	Keep a tested, automated rollback and a validated last-known-good state	T4	9 / 9	Slack 2021-09-30; Cloudflare 2022-06-21
C5	Test changes under production-representative data, scale thresholds and version mixes	T5	19 / 22	Cloudflare 2019-07-02; AWS 2020-11-25
C6	Route all changes, including manual and automated operations, through one governed pipeline; no silent bypass	T6	11 / 11	Cloudflare 2019-07-02; Meta 2021-10-04
C7	Add change-specific health signals for the failure mode the change could trigger	T7	5 / 7	Slack 2020-05-12; AWS 2020-11-25
C8	Make consumers fail safe: reject malformed input and keep the last good configuration	T8	4 / 5	Discord 2020-03-16; Cloudflare 2024-11-14
C9	Check capacity headroom and tested scaling limits as a deployment gate	T9	7 / 9	AWS 2020-11-25; AWS 2023-06-13
C10	Keep recovery paths independent of the system being changed (out-of-band access, independent dependencies)	T10	4 / 7	Roblox 2021-10-28; Azure 2025-10-09
C11	Maintain and regularly exercise kill switches and configuration freezes for fast containment	T4	14 / 14	Cloudflare 2019-07-02; Cloudflare 2020-07-17

C11 also counts change-triggered incidents contained by a kill switch or by disabling the offending component (recorded in the notes field).

Some of these controls can be applied immediately. Practitioners can audit every channel through which production can change, including configuration systems, data pipelines, feature-flag services, operating-system update feeds, administrative tools and manual commands, and check that each passes through staging, validation and audit (C1, C2, C6). Most organisations already stage code. The corpus suggests that the larger gains lie in the channels that are not code. For each channel, the consumer should be able to reject a bad artifact and continue on the last good one (C8). That last good state should be protected from being overwritten by an unproven change (C4). Finally, recovery tooling should be checked for dependence on the system it is meant to recover (C10), and kill switches should be exercised regularly (C11).

8 Threats to Validity

Construct validity. The codes reflect what the postmortems say, not necessarily what happened. Providers decide what to disclose, and the stated validation gap is the provider's own framing. Some fields are often not stated, including rollout scope (15 of 52 change-triggered incidents) and detection source (21 of 70), so proportions for these fields are computed over stated values only. For rollout scope, "not stated" is used both when the source is silent and, by the codebook, for

incidents not started by a change. We therefore analyse scope only within change-triggered incidents. Durations are not uniformly defined. Some sources measure to restoration of the main service and others to full data restoration, and conflicts were resolved in favour of the stated figure. The blast-radius level is the author's ordinal coding of free text and is released for re-coding.

Internal validity. The study is observational. Associations between scope, mitigation and duration may be confounded by provider, service type and incident severity. For example, the short durations of globally pushed changes are largely a property of one provider's edge incidents (Section 5.2). We make no causal claims, and we report the provider-excluded sensitivity analyses where they matter.

Reliability. All primary codes were assigned by a single coder. Our reliability check uses a rule-based coder, not a second human, and is likely to overstate agreement (Section 4.5). The thematic coding was also done by one person, mitigated by a released keyword mapping and a recorded justification for each of the 19 overrides. An independent re-coding of the dataset by other researchers would be the most valuable extension of this work, and the replication package is designed to make it easy.

External validity. The corpus is a purposive sample of 70 incidents. It over-represents providers that publish detailed postmortems (Cloudflare and Azure together contribute 30 incidents). It probably over-represents change-triggered incidents, because a change makes for a clear, publishable narrative. The proportions should therefore not be read as population rates of outage causes. They describe large, publicly explained outages. The sensitivity analysis excluding the two largest contributors leaves the change-related share essentially unchanged, but lowers the configuration share.

Conclusion validity. Groups are small (5 to 11 incidents per scope category), and after Holm correction across eleven tests no comparison remains significant at $\alpha = 0.05$. The smallest adjusted p-value is for the association between blast level and duration. We therefore emphasise effect sizes, counts and medians, and present the tests as exploratory. Effects of the size observed here would need larger corpora to confirm.

9 Conclusion

We coded 70 large outages from official postmortems of 22 cloud and SaaS providers. Changes started about three quarters of them, and configuration, increasingly in the form of machine-generated data, was the most frequent changed artifact. Global pushes were strongly associated with wide impact. Changes applied to a single site often reached far beyond it, and only about a third of change-triggered outages were ended by rollback. The safeguards that providers most often said were missing were representative testing, staged and health-gated rollout, and semantic validation of configuration and data. The last two were concentrated in configuration changes. We release the dataset, the coding files and the analysis code, and we offer a Safe Change Checklist in which every control is traced to the incidents that motivate it. The central practical lesson is simple to state and harder to implement: every path by which production can change needs the same staging, validation and containment that code already receives.

Statements and Declarations

Funding. No funding was received for conducting this study.

Competing interests. The author is employed by Microsoft Corporation, which operates Microsoft Azure; 12 of the 70 analysed incidents are public Azure post-incident reviews. This work was conducted independently, outside the author's employment and without funding; it uses only publicly available postmortems and no employer data, systems or confidential information, and it does not represent the views of the author's employer. The author declares no other competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethics approval. Not applicable. The study analysed only publicly available documents (vendor postmortems and published literature); it involved no human participants, no personal data and no interventions.

Consent to participate and consent for publication. Not applicable.

Clinical trial number: not applicable.

Data availability. The dataset generated and analysed during the current study (public_postmortems_dataset.csv with its README, listing the public source URL of every incident), together with the derived coding files, figures and results file (results.json), is provided

as supplementary material (paperD_code_data.zip). Upon acceptance these materials will be deposited in a public archival repository (Zenodo) with a DOI. All underlying postmortems are publicly available at the source URLs listed in the dataset. No proprietary or confidential data were used.

Code availability. The analysis code (analysis.py) is provided in the same supplementary archive (paperD_code_data.zip) and will be deposited with the data on Zenodo upon acceptance.

Author contributions. P.S.L.P. is the sole author. CRediT roles: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, and Writing – review & editing.

Use of AI tools. In line with Springer Nature's policy, the author discloses that an AI assistant (Claude, Anthropic) was used to help draft and edit the text and to write and debug the analysis code. The dataset was coded from public sources as documented in the replication package, and every reported number is produced by the released analysis script. The author designed the study, reviewed and verified all content, codes, code and results, and takes full responsibility for the article.

References

Beyer B, Jones C, Petoff J, Murphy NR (eds) (2016) Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media, Sebastopol, CA. Ch. 8, Release Engineering. https://sre.google/sre-book/release-engineering/

Braun V, Clarke V (2006) Using thematic analysis in psychology. Qualitative Research in Psychology 3(2):77–101. https://doi.org/10.1191/1478088706qp063oa

Bronson N, Aghayev A, Charapko A, Zhu T (2021) Metastable failures in distributed systems. In: Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '21). ACM, pp 221–227. https://doi.org/10.1145/3458336.3465286

Chu X, Talluri S, Lu Q, Iosup A (2025) An empirical characterization of outages and incidents in public services for large language models. In: Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE '25). ACM, pp 69–80. https://doi.org/10.1145/3676151.3719372

Cohen J (1960) A coefficient of agreement for nominal scales. Educational and Psychological Measurement 20(1):37–46. https://doi.org/10.1177/001316446002000104

Cruzes DS, Dybå T (2011) Recommended steps for thematic synthesis in software engineering. In: 2011 International Symposium on Empirical Software Engineering and Measurement (ESEM). IEEE, pp 275–284. https://doi.org/10.1109/ESEM.2011.36

Davidovič Š, Murphy NR, Kalt C, Beyer B (2018) Configuration design and best practices. In: Beyer B, Murphy NR, Rensin DK, Kawahara K, Thorne S (eds) The Site Reliability Workbook. O'Reilly Media, ch 14. https://sre.google/workbook/configuration-design/

Ganatra V, Parayil A, Ghosh S, Kang Y, Ma M, Bansal C, Nath S, Mace J (2023) Detection is better than cure: a cloud incidents perspective. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23). ACM, pp 1891–1902. https://doi.org/10.1145/3611643.3613898

Ghosh S, Shetty M, Bansal C, Nath S (2022) How to fight production incidents? An empirical study on a large-scale cloud service. In: Proceedings of the 13th Symposium on Cloud Computing (SoCC '22). ACM, pp 126–141. https://doi.org/10.1145/3542929.3563482

Gunawi HS, Hao M, Leesatapornwongsa T, Patana-anake T, Do T, Adityatama J, Eliazar KJ, Laksono A, Lukman JF, Martin V, Satria AD (2014) What bugs live in the cloud? A study of 3000+ issues in cloud systems. In: Proceedings of the ACM Symposium on Cloud Computing (SoCC '14). ACM, pp 1–14. https://doi.org/10.1145/2670979.2670986

Gunawi HS, Hao M, Suminto RO, Laksono A, Satria AD, Adityatama J, Eliazar KJ (2016) Why does the cloud stop computing? Lessons from hundreds of service outages. In: Proceedings of the 7th ACM Symposium on Cloud Computing (SoCC '16). ACM, pp 1–16. https://doi.org/10.1145/2987550.2987583

Holm S (1979) A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics 6(2):65–70. https://www.jstor.org/stable/4615733

Huang L, Magnusson M, Muralikrishna AB, Estyak S, Isaacs R, Aghayev A, Zhu T, Charapko A (2022) Metastable failures in the wild. In: 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). USENIX Association, pp 73–90. https://www.usenix.org/conference/osdi22/presentation/huang-lexiang

Huang P, Guo C, Zhou L, Lorch JR, Dang Y, Chintalapati M, Yao R (2017) Gray failure: the Achilles' heel of cloud-scale systems. In: Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS XVI). ACM, pp 150–155. https://doi.org/10.1145/3102980.3103005

Kerby DS (2014) The simple difference formula: an approach to teaching nonparametric correlation. Comprehensive Psychology 3:11.IT.3.1. https://doi.org/10.2466/11.IT.3.1

Kitchenham B, Madeyski L, Budgen D, Keung J, Brereton P, Charters S, Gibbs S, Pohthong A (2017) Robust statistical methods for empirical software engineering. Empirical Software Engineering 22(2):579–630. https://doi.org/10.1007/s10664-016-9437-5

Kruskal WH, Wallis WA (1952) Use of ranks in one-criterion variance analysis. Journal of the American Statistical Association 47(260):583–621. https://doi.org/10.1080/01621459.1952.10483441

Landis JR, Koch GG (1977) The measurement of observer agreement for categorical data. Biometrics 33(1):159–174. https://doi.org/10.2307/2529310

Li X, Yu G, Chen P, Chen H, Chen Z (2022) Going through the life cycle of faults in clouds: guidelines on fault handling. In: 2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE). IEEE, pp 121–132. https://doi.org/10.1109/ISSRE55969.2022.00022

Li Z, Cheng Q, Hsieh K, Dang Y, Huang P, Singh P, Yang X, Lin Q, Wu Y, Levy S, Chintalapati M (2020) Gandalf: an intelligent, end-to-end analytics service for safe deployment in large-scale cloud infrastructure. In: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20). USENIX Association, pp 389–402. https://www.usenix.org/conference/nsdi20/presentation/li

Liguori C (2020) Automating safe, hands-off deployments. Amazon Builders' Library. https://aws.amazon.com/builders-library/automating-safe-hands-off-deployments/

Liu H, Lu S, Musuvathi M, Nath S (2019) What bugs cause production cloud incidents? In: Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '19). ACM, pp 155–162. https://doi.org/10.1145/3317550.3321438

Luu D (n.d.) A list of post-mortems. GitHub repository. https://github.com/danluu/post-mortems. Accessed 5 October 2026

Mann HB, Whitney DR (1947) On a test of whether one of two random variables is stochastically larger than the other. The Annals of Mathematical Statistics 18(1):50–60. https://doi.org/10.1214/aoms/1177730491

Maurer B (2015) Fail at scale. ACM Queue 13(8):30–46. https://doi.org/10.1145/2838344.2839461

Mehta S, Bhagwan R, Kumar R, Bansal C, Maddila C, Ashok B, Asthana S, Bird C, Kumar A (2020) Rex: preventing bugs and misconfiguration in large services using correlated change analysis. In: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20). USENIX Association, pp 435–448. https://www.usenix.org/conference/nsdi20/presentation/mehta

Ralph P et al (2020) Empirical standards for software engineering research. arXiv:2010.03525. https://arxiv.org/abs/2010.03525

Runeson P, Höst M (2009) Guidelines for conducting and reporting case study research in software engineering. Empirical Software Engineering 14(2):131–164. https://doi.org/10.1007/s10664-008-9102-8

Sayagh M, Kerzazi N, Adams B, Petrillo F (2020) Software configuration engineering in practice: interviews, survey, and systematic literature review. IEEE Transactions on Software Engineering 46(6):646–673. https://doi.org/10.1109/TSE.2018.2867847

Sillito J, Kutomi E (2020) Failures and fixes: a study of software system incident response. In: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME). IEEE, pp 185–195. https://doi.org/10.1109/ICSME46990.2020.00027

Sun X, Cheng R, Chen J, Ang E, Legunsen O, Xu T (2020) Testing configuration changes in context to prevent production failures. In: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). USENIX Association, pp 735–751. https://www.usenix.org/conference/osdi20/presentation/sun

Tang C, Kooburat T, Venkatachalam P, Chander A, Wen Z, Narayanan A, Dowell P, Karl R (2015) Holistic configuration management at Facebook. In: Proceedings of the 25th Symposium on Operating Systems Principles (SOSP '15). ACM, pp 328–343. https://doi.org/10.1145/2815400.2815401

Tomczak M, Tomczak E (2014) The need to report effect size estimates revisited. An overview of some recommended measures of effect size. Trends in Sport Sciences 21(1):19–25. https://tss.awf.poznan.pl/The-need-to-report-effect-size-estimates-revisited-An-overview-of-some-recommended,188960,0,2.html

Virtanen P, Gommers R, Oliphant TE et al (2020) SciPy 1.0: fundamental algorithms for scientific computing in Python. Nature Methods 17(3):261–272. https://doi.org/10.1038/s41592-019-0686-2

Warner A, Davidovič Š, Hidalgo A, Beyer B, Smith K, Duftler M (2018) Canarying releases. In: Beyer B, Murphy NR, Rensin DK, Kawahara K, Thorne S (eds) The Site Reliability Workbook. O'Reilly Media, ch 16. https://sre.google/workbook/canarying-releases/

Wilson EB (1927) Probable inference, the law of succession, and statistical inference. Journal of the American Statistical Association 22(158):209–212. https://doi.org/10.1080/01621459.1927.10502953

Xu T, Zhou Y (2015) Systems approaches to tackling configuration errors: a survey. ACM Computing Surveys 47(4):70:1–70:41. https://doi.org/10.1145/2791577

Xu T, Jin X, Huang P, Zhou Y, Lu S, Jin L, Pasupathy S (2016) Early detection of configuration errors to reduce failure damage. In: 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16). USENIX Association, pp 619–634. https://www.usenix.org/conference/osdi16/technical-sessions/presentation/xu

Yin Z, Ma X, Zheng J, Zhou Y, Bairavasundaram LN, Pasupathy S (2011) An empirical study on configuration errors in commercial and open source systems. In: Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP '11). ACM, pp 159–172. https://doi.org/10.1145/2043556.2043572

Yuan D, Luo Y, Zhuang X, Rodrigues GR, Zhao X, Zhang Y, Jain PU, Stumm M (2014) Simple testing can prevent most critical failures: an analysis of production failures in distributed data-intensive systems. In: 11th USENIX

Symposium on Operating Systems Design and Implementation (OSDI 14). USENIX Association, pp 249–265. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/yuan

Zhang Y, Yang J, Jin Z, Sethi U, Rodrigues K, Lu S, Yuan D (2021) Understanding and detecting software upgrade failures in distributed systems. In: Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21). ACM, pp 116–131. https://doi.org/10.1145/3477132.3483577